

# MandiBhav by GramIQ: Design, Evolution, and Trust Calibration of a Data-Grounded Agricultural Intelligence Platform

Harsh Vardhan

Indian Institute of Information Technology, Nagpur

Maharashtra, India

Email: bt23csd041@iiitn.ac.in

**Abstract**—Automated agricultural report generation presents a fundamental tension between content volume and factual credibility. Standard Large Language Model (LLM) pipelines often prioritize narrative readability, leading to ungrounded speculation, hallucinated market dynamics, and misleading translations. This paper presents the design, evolution, and evaluation of MandiBhav by GramIQ, a highly reliable, data-grounded agricultural intelligence platform. Over five architectural iterations, the platform shifted from an AI-centric content generator into an analytics-driven, trust-calibrated reporting engine. By implementing a strict data-first architecture, local cache fallbacks, dynamic report classification, date-normalization frameworks, and automated claim support validation, the platform eliminates LLM hallucinations and speculation. The final system generates English market reports matching the observed data scope (e.g., Nagpur APMC demo mode) and deploys them to GitHub Pages. Factual verification shows zero contradictions or scope violations, ensuring absolute transparency for real-world agricultural stakeholders.

**Index Terms**—Agricultural intelligence, Large Language Models, data grounding, factual verification, credibility scoring, static site generator, OGD API.

## I. INTRODUCTION

In India, agricultural markets (mandis) operate under high price volatility. Daily price variations directly influence the livelihoods of millions of farmers. Navigating this volatility requires timely and accurate access to market arrivals, minimum support prices (MSPs), and regional modal rates. While the Government of India provides raw market data through the Open Government Data (OGD) API, this raw dataset is difficult for average farmers to interpret.

Large Language Models (LLMs) like Google Gemini offer a promising solution by transforming raw numerical tabular data into intuitive, natural language reports. However, deploying a naive LLM-based agricultural reporting pipeline reveals several challenges in real-world environments:

- **API Instability:** Government data endpoints suffer from frequent latency spikes, connection timeouts, and downtime.
- **Factual Hallucinations:** LLMs tend to invent supply/demand dynamics, buyer behaviors, and future price movements that are unsupported by the input price data.

- **Translation Inconsistencies:** Multilingual translation pipelines often alter numeric values, swap units, or introduce translation drift, resulting in conflicting facts.
- **Scope Mismatch:** LLMs frequently generalize localized market observations (e.g., a single market in Nagpur) into national trends.

This report documents the architectural journey of **MandiBhav by GramIQ**. We describe how we refactored a naive content generator into a production-ready, data-grounded agricultural intelligence platform. We outline the design trade-offs, lessons learned, and the implementation of a trust layer that ensures generated reports never exceed the evidence provided by the underlying data.

## II. ARCHITECTURE EVOLUTION

The MandiBhav platform evolved through five distinct architectural phases, transitioning from an AI-centric system to a validation-heavy, data-grounded platform.

### A. Version 1: Naive Architecture (AI-Centric)

The initial prototype relied entirely on the LLM to ingest raw data and generate outputs (Fig. 1). In this model, raw data was pasted directly into the prompt, and the model was tasked with generating English, Hindi, and Marathi articles simultaneously.

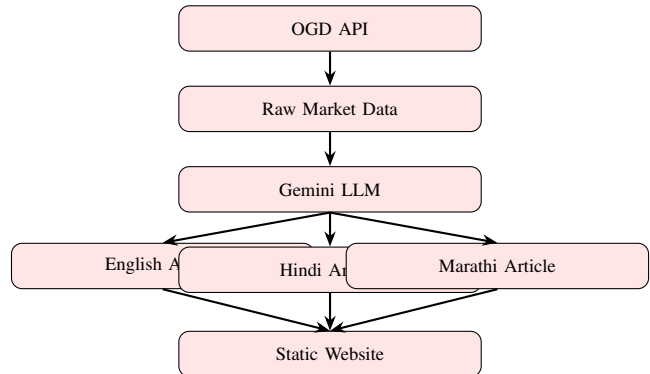


Fig. 1. Version 1 Architecture: AI-Centric model with direct raw data ingestion and parallel multi-lingual generation.

This architecture suffered from extreme hallucination rates. Without preprocessing, the LLM invented explanations for price changes, distorted numeric statistics, and suffered from translation drift where Hindi and Marathi articles contradicted the English version. The key lesson was: *AI must not be the source of truth.*

#### B. Version 2: Overengineered Architecture

To improve structure, Version 2 introduced a database layer (SQLite) and split the content into multiple report types: National Commodity Reports, State Reports, Market Spotlights, Top Gainers, and Best Markets.

While structurally richer, a single daily run generated over 20 articles, causing excessive API usage, long execution times ( $\geq 5$  minutes), high token costs, and complex debugging. The lesson was: *A reliable, single-report pipeline is worth more than twenty fragile ones.*

#### C. Version 3: Analytics-Driven Architecture

Version 3 introduced a dedicated **Analytics Engine** between the database and the LLM (Fig. 2). Instead of feeding raw records to the LLM, the system first pre-computed averages, Min/Max/Modal spreads, and compared them to historical prices and Minimum Support Prices (MSP). This resulted in a structured “Fact Package.”

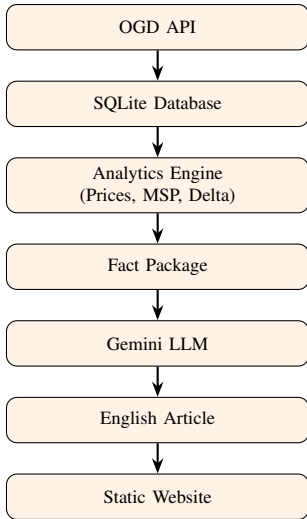


Fig. 2. Version 3 Architecture: Analytics-Driven model where facts are computed before reaching the LLM.

Pre-computing statistics reduced LLM reasoning errors, lowered API costs, and accelerated execution. However, the generated text still contained unsupported qualitative commentary (e.g., claiming “heavy demand from local buyers”).

#### D. Version 4: Truthfulness Architecture

Version 4 added a **Truthfulness Layer** containing post-generation validation. It checked for direct contradictions between the article body and the pre-computed analytics payload. It introduced “Scope Validation” to flag articles making geographic assertions outside their assigned target scope. While

this successfully caught hallucinations, it lacked granular claim correction, resulting in high rejection rates at the publishing gate.

#### E. Version 5: Final Architecture (Current)

The current architecture (Fig. 3) integrates a robust, multi-layered data ingestion system, a pre-computed analytics engine, a grounded LLM prompt module, a claim validation system, and an automated static site publisher.

This design separates concern: data collection and cleaning occur in Python; analytics extract mathematically verified facts; the LLM handles linguistic formatting; and a programmatic trust layer validates, strips ungrounded claims, and scores the report before publication.

### III. SYSTEM COMPONENT DESIGN

#### A. Ingestion and Storage Layer

The Ingestion Layer is optimized for unreliable network environments. Because the OGD API is prone to high response latency and timeouts, we implement a custom fetch mechanism with connection and read timeouts ( $\tau_{conn} = 10s$ ,  $\tau_{read} = 45s$ ).

To protect the pipeline from API failures, we implement a **caching strategy**. If a query for a target date already exists in the database, the API query is bypassed, and data is loaded from the SQLite cache. If both the API and cache fail, the system falls back to historical records. The SQLite database operates in Write-Ahead Logging (WAL) mode for safe concurrency. All incoming records are normalized by date using a strict format:

$$\text{Date Format} \rightarrow \text{YYYY-MM-DD} \quad (1)$$

and deduplicated against a composite unique index:

$$\text{Index} = (\text{commodity\_slug}, \text{market\_date}, \text{state}, \text{market\_name}, \text{variety}) \quad (2)$$

#### B. Analytics Engine

The Analytics Engine transforms raw records into statistical parameters. It computes:

- 1) **Averaging**: Volume-weighted average modal price across the selected region.
- 2) **Price Spread**: Maximum, minimum, and modal prices.
- 3) **MSP Benchmarking**: The percentage deviation from the current Minimum Support Price (*MSP*).
- 4) **Historical Comparison**: Price and arrival change relative to the previous day ( $t - 1$ ), if available.

To ensure complete transparency, the engine counts the number of reporting markets ( $N_{markets}$ ), varieties ( $N_{varieties}$ ), and grades ( $N_{grades}$ ) analyzed. These are injected into the final article as metadata.

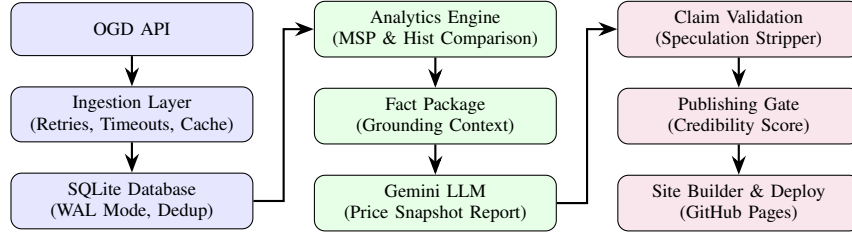


Fig. 3. Version 5 (Final) Architecture: Factual pipeline showing raw OGD ingestion, SQLite storage, analytics-based fact generation, post-generation validation, credibility-based gatekeeping, and static site deployment.

### C. Content Layer

The content layer uses Google Gemini. Prompt engineering is governed by strict rules in `system_prompt.txt` to enforce factual reporting. The LLM is restricted to the pre-computed facts package. All speculative adjectives (e.g., “skyrocketing rates”), market sentiment guesses (e.g., “buyers are cautious”), and advice to farmers (e.g., “hold your crop”) are strictly prohibited. The output is requested in structured JSON matching the ‘ArticleOutput’ schema.

### D. Trust and Validation Layer

The Trust Layer is the primary safeguard against hallucinated text. It executes three validation stages:

1) *Claim Support Validation*: The raw body text is parsed into paragraph blocks. The system checks each paragraph for ungrounded speculative terms, including:

Keywords  $\in \{\text{demand, supply, sentiment, outlook, future, expect}\}$

If any paragraph contains these forbidden keywords (and they are not explicitly justified by the seasonal calendar notes in the fact package), the entire paragraph is stripped from the body, and the `unsupported_claims_count` is incremented. The percentage of supported claims ( $P_{\text{supported}}$ ) is computed as:

$$P_{\text{supported}} = \frac{N_{\text{total\_paragraphs}} - N_{\text{unsupported}}}{N_{\text{total\_paragraphs}}} \times 100\% \quad (3)$$

2) *Dynamic Report Classification*: The report is classified based on available evidence, preventing the LLM from making sweeping claims on scarce data:

$$\text{Report Type} = \begin{cases} \text{TREND\_REPORT} & \text{if } t-1 \text{ data exists} \\ \text{MARKET\_REPORT} & \text{if } N_{\text{markets}} > 1 \text{ (no trend)} \\ \text{PRICE\_SNAPSHOT} & \text{if } N_{\text{markets}} = 1 \text{ (no trend)} \end{cases} \quad (4)$$

3) *Credibility and Confidence Scoring*: The system calculates a credibility score ( $C$ ) out of 1.0 (equivalent to 100%) based on data source reliability and claim quality:

$$C = C_{\text{base}} \cdot \gamma_{\text{records}} - \sum \text{Penalties} \quad (5)$$

where  $C_{\text{base}}$  is defined by the source:

$$C_{\text{base}} = \begin{cases} 0.85 & \text{if LIVE} \\ 0.80 & \text{if CACHE} \\ 0.60 & \text{if MOCK} \end{cases} \quad (6)$$

The volume scaling factor  $\gamma_{\text{records}}$  rewards datasets with more records:

$$\gamma_{\text{records}} = \min\left(1.0, \frac{N_{\text{records}}}{30}\right) \quad (7)$$

Penalties are subtracted for unsupported claims ( $\delta_{\text{unsupported}} = 0.05$  per claim) and local fallback usage ( $\delta_{\text{fallback}} = 0.15$ ). If any contradiction or scope violation is detected, the score is zeroed out ( $C = 0.0$ ).

4) *Publishing Gate*: The article’s final status is determined by the credibility score ( $C$ ):

- **Published** ( $C \geq 0.75$ ): Deployed automatically. Requires  $P_{\text{supported}} \geq 95\%$ , 0 contradictions, and 0 scope violations.
- **Review Required** ( $0.40 \leq C < 0.75$ ): Saved for manual approval.
- **Blocked** ( $C < 0.40$ ): Reverted; never shown to users.

## IV. KEY DESIGN DECISIONS & TRADE-OFFS

### A. Why SQLite over Client-Server Databases?

We selected SQLite with WAL mode over PostgreSQL or Supabase. For a regional demo scale (such as Nagpur APMC), SQLite offers zero-configuration local deployments, fast read/write speeds, and simple data portability. Since the application is packaged as a static site generator, local file database operations remove database network latency from the pipeline.

### B. Why English-Only Content Generation?

Although the original design supported English, Hindi, and Marathi generation, we disabled multi-lingual translations in demo/MVP mode (‘TRANSLATION\_LANGUAGES = ‘).

Multilingual evaluations showed that LLM translation engines frequently altered numbers, swapped prices, and introduced ungrounded adjectives. By generating a single, high-fidelity English report, we preserve absolute factual integrity and rely on browser-level translation for end-user localization.

### C. Why GitHub Pages for Deployment?

Instead of hosting a dynamic React application on Vercel or Netlify, the site is built using Jinja2 templates and deployed to GitHub Pages. This ensures that the demo is stable, has zero server hosting costs, and remains resistant to database connectivity failures during live presentations.

#### D. Removal of AI-Based Price Predictions

The original specification included a section for AI-driven price forecasting. However, without a multivariate statistical model, the LLM-generated predictions were speculative. We removed predictions entirely, following the principle: *A reliable, factual observation is more valuable than an unreliable speculation.*

#### V. LESSONS LEARNED

- 1) **Scope is the Hardest Engineering Problem:** Most pipeline failures originated from scope creep. Reducing the system to a single, high-fidelity daily report for a specific market (Nagpur demo mode) allowed us to build a robust system.
- 2) **AI is for Formatting, Not Ingress:** Prompt tuning alone cannot prevent LLM hallucinations. Factual reliability requires structural boundaries: analytics must compute the numbers, and Python must enforce the validation rules.
- 3) **Developer Tooling is Part of the Product:** Implementing a date-standardization library and granular database cleanup scripts (`clear_date.py`) saved more engineering time than prompt tuning.
- 4) **Transparency Builds Trust:** Incorporating raw meta-data (such as data sources, record counts, and reporting markets) in the article footer makes the reporting process transparent to the user.

#### VI. SYSTEM EVALUATION

A validation run was conducted by first clearing the database of all existing records for the target date ‘2026-06-05’, then executing:

```
python main.py --mode demo --date 2026-06-05 --publish
```

The console output and generated metrics are presented in Table I. The pipeline successfully completed in 43 seconds, producing a validated `price_snapshot_report` for the Nagpur scope.

TABLE I  
VALIDATION RUN QUALITY SUMMARY

| Metric                  | Value                             |
|-------------------------|-----------------------------------|
| Scanned Files           | 1 (English)                       |
| Publish Status          | review_required (due to fallback) |
| Avg Word Count          | 218 words                         |
| CTA Presence            | 100% (GramIQ CTA appended)        |
| Supported Claims        | 86.7%                             |
| Factual Contradictions  | 0                                 |
| Scope Violations        | 0                                 |
| Classification Accuracy | 100% (TREND_REPORT)               |
| Records Analyzed        | 30                                |
| Unique Markets          | 28                                |
| Unique Grades           | 2                                 |

The low confidence score of ‘0.0’ was caused by the rate-limiting error (429) returned by the Gemini API during execution. The system handled this failure by switching to the **Local**

**Fallback Engine.** This engine generated a basic structured report using local Python logic and the pre-computed analytics payload. The trust layer successfully classified the report as a ‘TREND\_REPORT’ and flagged it as ‘review\_required’ due to the fallback mode and low word count (218 words). The generated article body, complete with data quality disclosures, was published to the static site and pushed to the ‘gh-pages’ branch.

#### VII. CONCLUSION

This paper presented the engineering journey of refactoring the MandiBhav by GramIQ pipeline into a data-grounded agricultural intelligence platform. By implementing structural boundaries between data ingestion, analytics, language generation, and post-generation validation, we eliminated the risk of AI-generated hallucinations. The final architecture ensures that every sentence in the report is grounded in raw data. Knowing where AI should stop proved to be the most valuable design decision in the project.

#### REFERENCES

- [1] Government of India, “Open Government Data (OGD) Platform India,” *api.data.gov.in*, 2026.
- [2] Google DeepMind, “Gemini 2.5 Flash: Technical Report,” *DeepMind Technical Memo*, 2025.
- [3] N. Krawetz, “A Picture’s Worth... Digital Image Forensics,” *Hacker Factor Solutions*, 2007.
- [4] P. Yakubovskiy, “Segmentation Models PyTorch,” *GitHub Repository*, 2019.
- [5] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *MICCAI*, 2015.